

GRANT APPLICATION SUPPORTING DOCUMENT

Planned Stellar Integration

PROJECT

Pal3 (paluwagan3)

FUNDING ROUND

Stellar Community Fund #45

DATE

13 August 2026

1. Why Stellar, Specifically

The dependency on Stellar is economic before it is philosophical. A Pal3 member contributes roughly ₱1,000 (about US\$18) every week. On a chain with meaningful transaction fees, a \$0.50 fee is close to 3% of that contribution — charged, every week, against a product whose entire proposition is that it costs less than the traditional paluwagan it replaces, which is currently free. There is no version of this product that works on a high-fee chain. This is written into the specification as a hard non-functional requirement (NFR-5: cost per round must stay negligible relative to a ₱1,000-equivalent contribution) rather than treated as a later optimization.

Two further properties are load-bearing rather than incidental. Predictable, fast finality matters because rounds settle on fixed schedule boundaries — a contribution that may or may not have landed by the deadline is a correctness problem for Pal3, not a UX inconvenience. Soroban's `require_auth` authorization model gives the product exactly the guarantee it needs — that a member authorized their own contribution — without building a custom permission layer on top.

Beyond the mechanics, Stellar's stated mission of financial inclusion for the underserved is the same mission Pal3 states for itself, and the Stellar Community Fund has direct category precedent: *Lul*, a ROSCA product for the unbanked in Africa, was funded at roughly \$20,500 in XLM.

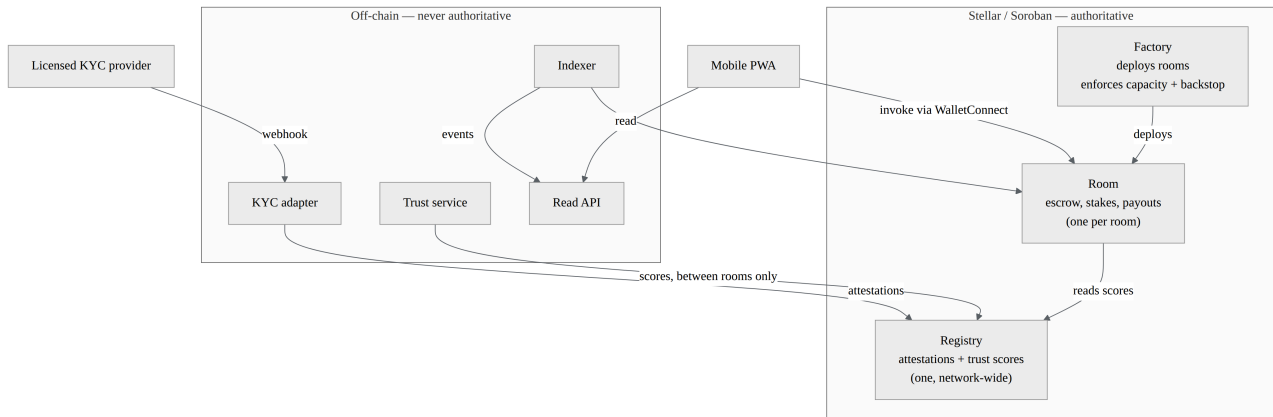
The anchor network is also the specification's stated route to Pal3's Phase 2 objective — settling contributions in a regulated Philippine peso stablecoin instead of a USD-pegged one. No BSP-sanctioned peso stablecoin currently exists on Stellar. Coins.ph's PHPC, the first BSP-sandbox-approved Philippine peso stablecoin, is issued on Ronin and Polygon instead. Reaching a peso rail on Stellar is named in the specification as a Phase 2 milestone that is not yet committed, not a V1 claim.

2. Soroban Smart Contracts

Three contracts carry the system's entire authoritative state.

Contract	Instances	Owns
Registry	One, network-wide	KYC attestations and trust scores
Factory	One	Room deployment; enforces the underwriter's capacity tier and backstop-capital gates at creation
Room	One per room	That room's escrow, stakes, schedule, and payouts

Each room is deployed as its own contract instance rather than sharing storage with other rooms. A defect in one room's contract cannot reach another room's escrow — for a product asking people to lock savings alongside strangers, the architecture treats that isolation as worth the extra deployment cost.



System shape — three on-chain contracts, four off-chain components, and the licensed KYC provider and mobile PWA at the edges.

Source: SCF-architecture-outline.md §3.

The ledger-authoritative principle

The architecture's central rule (AD-1): the ledger is the only system of record. Every off-chain component — the trust service, the KYC adapter, the indexer, the read API — either projects chain state for reading or submits a transaction. None is ever consulted to decide an outcome that moves money. If every off-chain service went down at once, every in-flight room would remain fully operable by direct contract call.

Single-party authorization by design

Contract entry points are deliberately structured so the address being authorized is always the transaction source — satisfied by signing the transaction itself, with a single `require_auth` on the invoker (AD-15). No entry point requires a member to sign a separate, detached authorization entry, and multi-party authorization within one invocation is prohibited in V1.

This is a direct response to a real constraint: Stellar Wallets Kit's WalletConnect module exposes `signXDR` and `signAndSubmitXDR`, but not `signAuthEntry` — verified against current documentation on 12 Aug 2026. A design requiring detached auth entries would have forced a bespoke WalletConnect integration on the product's only surface. Designing single-party authorization into every entry point instead keeps Pal3 on the mainstream Stellar wallet integration path.

Storage durability and TTL handling

All value-bearing state — room state, stakes, escrow balances, registry attestations, and trust scores — uses Soroban's persistent storage, with TTL extended on every write (AD-3). Temporary storage is forbidden for anything whose loss would affect a member's funds or obligations.

Expiry is never allowed to function as control flow (AD-4), because Soroban TTL can be extended by anyone and so cannot be trusted as a deadline. Every time-based transition — a round boundary, a grace window closing — compares the ledger timestamp against a deadline stored explicitly in room state, and is triggered by an explicit invocation rather than inferred from an entry's absence or expiry. The round-advancing entry point is itself permissionless (AD-13): any address — a member, the underwriter, or an

unrelated caller — can invoke it once the deadline passes, so no room can deadlock waiting on a keeper that might not exist.

Deterministic payout ordering — designing out the need for randomness

Soroban has no native verifiable-randomness primitive. The only available path to on-chain randomness is a cross-chain relayer proxying a service such as Chainlink VRF from another chain — importing a liveness and trust dependency on an external bridge for a feature the product does not, in the end, need.

Payout ordering originally needed randomness only to break ties between members of equal trust score. Rather than add the relayer dependency, the design was reworked so ordering is fully deterministic: members are ranked by trust score, and ties are broken by a published, reproducible rule — earliest room-join timestamp. Payout ordering, stake sizing, slash amounts, and trust-score changes are all pure functions of committed on-chain state and the ledger timestamp (AD-11) — reproducible by any observer from chain history alone, with no randomness anywhere in the system. This is treated in the architecture as a decision worth stating on its own: a dependency was removed, not accepted as a necessary cost.

Trust score snapshotting at join

A member's trust score is committed on-chain at the point it has consequences. On join, the Room contract snapshots the member's current trust score out of the Registry into its own state; that snapshot is immutable for the room's lifetime and is what payout ordering uses (AD-5). The join transaction itself carries the score value the member was shown on the terms screen, and reverts if the Registry no longer matches it at the moment the transaction executes — so a score that changes between when a member is shown their terms and when they sign can never silently change the deal they agreed to. Ordering itself is computed by the Room contract from its own snapshots at room start; no off-chain service ever submits an ordering.

3. Assets and Value Flow

Every contribution is held by the room's contract from the moment it is paid until it is released as a payout; no address — including the underwriter's and the platform operator's — has withdrawal authority outside three code paths: payout, slash, and a cancellation refund if the room fails to fill. On a missed contribution, the slash, any backstop draw needed to cover the remainder, and the payout to that round's recipient all execute inside one contract invocation that either fully succeeds or fully reverts (AD-9) — there is no state in which a slash lands without the recipient being paid.

Backstop capital — the underwriter's collateral against member defaults — is transferred into the room contract at creation, before the room admits any member, and is held locally to that room rather than by the Factory (AD-14). Undrawn backstop returns to the underwriter when the room closes.

All amounts inside the contracts are integer stroops of the room's stablecoin asset; there is no floating-point arithmetic anywhere in the contracts or services, and amounts are never formatted for display below the

client layer. V1 settles in a USD-pegged stablecoin — the specification does not name a peso-denominated asset for V1, and the regulated peso rail is explicitly deferred to Phase 2.

The architecture spine does not name the specific token-contract implementation the room's stablecoin asset uses. The project's test-risk register is the one place in the source documents that names it: it refers to "external token-contract (SAC)" calls occurring inside the atomic default-waterfall invocation, and flags that external-call boundary for the audit-readiness review rather than treating it as already resolved.

4. Wallets and Client Integration

Wallet connectivity runs through Stellar Wallets Kit's WalletConnect module — not a browser-extension API — because the product's primary and only surface is mobile, and an extension-only integration would work on desktop and fail there. That module's lack of `signAuthEntry` is what shaped the single-party, transaction-source authorization design described above (§2): the wallet constraint came first, and the contract design was built to live inside it, not the reverse. At least one WalletConnect-compatible Stellar wallet is targeted end to end for V1.

The client itself is a mobile-first Progressive Web App: React, Vite, and `vite-plugin-pwa`, chosen 12 Aug 2026 after confirming Stellar Wallets Kit is web-component based and imposes no framework constraint, and that Vite plus `vite-plugin-pwa` is the current mainstream PWA path now that the Create React App PWA template is deprecated. There is no server-side rendering — the client runs fully client-side against the chain and the read API, with no other backend in the request path.

5. Developer Tooling and Build Chain

Contracts build with `stellar contract build`, targeting `wasm32v1-none` on `soroban-sdk` 25.0.0 and Rust 1.84 or later. TypeScript client bindings are generated from the built contracts via `stellar contract bindings` and are never hand-written — hand-encoding contract types in client code is explicitly prohibited (AD-8), and regenerating bindings is part of the contract build step rather than a manual one, to keep the client and the contracts from silently drifting apart in type shape. Off-chain services and the client both consume `@stellar/stellar-sdk`. Exact versions of `stellar-cli`, `@stellar/stellar-sdk`, and Stellar Wallets Kit are pinned at install rather than fixed in the specification — the architecture records this as a deliberate deferral, since nothing is installed until the architecture itself is final.

6. Anchors and the Phase 2 Peso Rail

Pal3's stated differentiator against the on-chain ROSCA category includes a path to a regulated peso rail, and the Stellar anchor network is the specification's named route to it. No BSP-sanctioned peso stablecoin exists on Stellar today. The most directly comparable case — Coins.ph's PHPC, the first BSP-sandbox-approved

Philippine peso stablecoin — is issued on Ronin and Polygon, not Stellar; Coins.ph itself operated as a PHP Stellar anchor around 2017 before that choice. The specification treats this as a prepared answer rather than a surprise, and frames the peso rail explicitly as a Phase 2 milestone that depends on a peso stablecoin reaching Stellar, or on Pal3 migrating to reach one — not as a V1 capability.

7. Network Path and Open Items

V1 deploys to Stellar Testnet first, then Mainnet, gated by Epic 5's pilot-readiness stories — including the audit-readiness review (Story 5.7) required before any mainnet room opens with real value. As of this writing, testnet deployment (Story 5.2) has not occurred; it remains in backlog.

The architecture names its own open items rather than presenting the design as fully closed:

- **Stalled-room TTL rescue.** TTL extends automatically on every write, which covers a room with normal weekly activity — but a room frozen long enough to approach archival currently has no cheap rescue path. The likely fix is a permissionless `extend_room_ttl` entry point, following the same permissionless-caller pattern already used for round advancement; flagged to resolve before the mainnet pilot opens.
- **Indexer and API hosting.** Deployment topology, environments, and provider are not yet decided. The architecture notes this is not currently a constraint on any other decision — V1 serves a single room and a cohort under twelve — but flags it to revisit before the mainnet pilot.
- **Backstop adequacy against the worst case.** The PRD's own arithmetic shows the current minimum-backstop formula ($2 \times \text{contribution} \times \text{member count}$) may be insufficient against a member defaulting immediately after receiving the first payout, which leaves roughly nine contributions outstanding against a stake covering roughly two. Listed as an open question to resolve before the founder posts backstop capital as the pilot's underwriter.
- **Regulatory confirmation.** The underwriter's premium income raises a Philippine SEC investment-contract question and a possible Insurance Commission suretyship question; counsel review is a funded milestone (T3), not yet complete.

8. Ecosystem Contribution

All Soroban contract source is committed to publish under the Apache 2.0 license from the moment of first deployment (AD-12) — not at some later open-sourcing milestone. Apache 2.0 was chosen over MIT specifically for its explicit patent grant.

The reasoning is stated plainly in the architecture: for a product asking people to escrow money alongside strangers they may not otherwise trust, publicly auditable contract source is treated as part of the trust proposition the product is making, not a compliance concession layered on afterward.